



Assignment 01

Subject: Programming for AI
Total Marks: 40

Post Date: 22-Oct-2025
Due Date: 31-Oct-2025

Course Instructor | Talha Shahid

Instructions

Please follow these instructions carefully to ensure your assignment is graded. Failure to adhere to these guidelines may result in penalties.

1. Deadline and Late Policy

Late Submissions: Late submissions are not permitted. Any assignment submitted after the deadline will automatically receive zero marks. Technical difficulties are not a valid excuse for late submission, so please plan to submit well before the deadline.

2. Academic Integrity: Plagiarism

- Your submission must be entirely your **own work**.
 - **Plagiarism** is strictly prohibited. This includes **copying code** from **classmates**, and **AI** (you can take guidance).
 - All submissions will be **checked** for **plagiarism**.
 - Any instance of plagiarism will result in an **automatic zero marks**.
-

3. Required Files

- You must submit a total of **05** files:
 - **One PDF Document:** A single, well-formatted PDF, renamed with your roll number(24K-####), containing the solutions for all questions. (See formatting details below).
 - **Four Code Files(.ipynb):** A separate, runnable code file for each question.
 - Please name your files clearly (e.g., **Q1.ipynb**, **Q2.ipynb**, **Q3.ipynb**, **Q4.ipynb**).
-

4. PDF Formatting Instructions

- Your single PDF document is your **main report**. It must be organized and clearly presented. For each question, you must include the following items in order:
 - Solution Code (**text format**):
 - Copy and paste your final, working code from your code file directly into the PDF.
 - **Do not use screenshots of your code.**
-



National University of Computer & Emerging Sciences, Karachi

School of Computing – BAI-3A

Fall 2025

- You must format the code properly so it is readable
-

Output Screenshot:

Immediately following the code for that question, insert a clear screenshot that shows your program running and producing the required output.

Submission Checklist:

Before you submit, check that you have:

- [] 1 formatted PDF file.
 - [] 4 separate code files (one for each question).
 - [] Checked that the PDF contains both the formatted code text and the output screenshot for every question.
 - [] Confirmed your work is original and free of plagiarism.
 - [] Submitted all 5 files before the deadline.
-

Submit the same on your [GitHub](#) also



Question 01:

A Pakistani e-commerce startup, "**KarachiDeals**," needs a "Customers Who Bought This Also Bought" feature. You are given a log of transactions and must process this data to find which products are most frequently purchased together.

Topics: Data Structures (Lists, Dictionaries, Sets, Tuples), Functions, Loops, zip, enumerate.

```
# The raw log of all items in all orders
transactionLog = [
    {'orderId': 1001, 'customerId': 'cust_Ahmed', 'productId': 'prod_10'},
    {'orderId': 1001, 'customerId': 'cust_Ahmed', 'productId': 'prod_12'},
    {'orderId': 1002, 'customerId': 'cust_Bisma', 'productId': 'prod_10'},
    {'orderId': 1002, 'customerId': 'cust_Bisma', 'productId': 'prod_15'},
    {'orderId': 1003, 'customerId': 'cust_Ahmed', 'productId': 'prod_15'},
    {'orderId': 1004, 'customerId': 'cust_Faisal', 'productId': 'prod_12'},
    {'orderId': 1004, 'customerId': 'cust_Faisal', 'productId': 'prod_10'},
]

# The mapping of product IDs to names
productCatalog = {
    'prod_10': 'Wireless Mouse',
    'prod_12': 'Keyboard',
    'prod_15': 'USB-C Hub',
}
```

Tasks:

Transform Data: Write a function `processTransactions(transactionsList)`. To efficiently manage the unique products for each customer, this function should return a dictionary where keys are `customerIds` and values are a set of their purchased `productIds`.

Find Pairs: Write a function `findFrequentPairs(customerData)`. This function needs to count how many times any two products were bought together. To track these co-purchases, you'll need a dictionary. For the keys, think about how to represent a pair of products as a single, immutable, and order-independent key (e.g., `('prod_10', 'prod_12')` should be the same as `('prod_12', 'prod_10')`).

Get Recommen: Write a function `getRecommendations(targetProductId, frequentPairs)`. This should iterate through your pairs map and find all pairs that include the `targetProductId`. It must return



a ranked list of the other products, sorted by their co-purchase count (highest first).

Generate Report: Write a function `generateReport(targetProductId, recommendations, catalog)`. Use `zip` and `enumerate` to align the ranked recommendation data with their product names from the catalog to print a clean, 1-indexed report.

Question 02:

A marketing firm wants to flag negative posts about their brand, "GulPhone." You need to process a list of posts, clean the text, score their sentiment, and find trending negative topics.

Topics: Functions, Data Structures (Lists, Dictionaries, Sets), List Comprehensions, `map`, `lambda`.

```
# The stream of posts to analyze
allPosts = [
    {'id': 1, 'text': 'I LOVE the new #GulPhone! Battery life is amazing.'},
    {'id': 2, 'text': 'My #GulPhone is a total disaster. The screen is already broken!'},
    {'id': 3, 'text': 'Worst customer service ever from @GulPhoneSupport. Avoid this.'},
    {'id': 4, 'text': 'The @GulPhoneSupport team was helpful and resolved my issue. Great service!'},
]

# Keywords and characters for processing
PUNCTUATION_CHARS = '!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
STOPWORDS_SET = {'i', 'me', 'my', 'a', 'an', 'the', 'is', 'am', 'was', 'and', 'but', 'if', 'or', 'to', 'of', 'at', 'by', 'for', 'with', 'this', 'that'}
POSITIVE_WORDS_SET = {'love', 'amazing', 'great', 'helpful', 'resolved'}
NEGATIVE_WORDS_SET = {'disaster', 'broken', 'worst', 'avoid', 'bad'}
```

Tasks:

Clean Text: Write a reusable function `preprocessText(text, punctuationList, stopwordsSet)`. This function must normalize the text by converting it to lowercase, removing all noise (characters in `punctuationList`), and filtering out low-value words (words in `stopwordsSet`).



Score Sentiment: Write a function `analyzePosts(postsList, punctuation, stopwords, positive, negative)`. To apply your `preprocessText` function to every post efficiently, use `map`. You can use a lambda function inside `map`. After processing, score each post (e.g., `+1` for each `POSITIVE_WORDS_SET`, `-1` for each `NEGATIVE_WORDS_SET`). Return a new list where each element is a dictionary containing the original `id`, `text`, the `processedText` list, and the final score.

Flag Posts: Write a function `getFlaggedPosts(scoredPosts, sentimentThreshold=-1)`. Use a single-line list comprehension to create a new list containing only the posts at or below the `sentimentThreshold`.

Find Topics: Write a function `findNegativeTopics(flaggedPosts)`. Iterate only through the list of negative posts from Task 3 to find all hashtags (#) and mentions (@). Return a dictionary that counts the frequency of each negative topic.

Question 03:

A logistics company, "LahoreLogistics," needs a simulation for its drone delivery system. You must model this system using OOP, focusing on how classes interact and protect their data.

Topics: OOP (Classes, Encapsulation, Composition, Aggregation).

Tasks:

Entity Classes: Create a `Package` class (with `packageId`, `weightInKg`) and a `Drone` class (with `droneId`, `maxLoadInKg`).

- A `Drone` must protect its internal state. Give it a "`private`" `_status` attribute. This attribute should only be changeable via public methods: `getStatus()` and `setStatus(newStatus)`. The setter must validate that the `newStatus` is one of (`'idle'`, `'delivering'`, `'charging'`).
- A `Drone` uses or carries a `Package` for a temporary delivery. The `Package` exists independently of the drone. Implement `Drone.assignPackage(packageObj)` to create this temporary link. This method should only succeed if the drone's status is `'idle'` and the package's weight is within the drone's `maxLoadInKg`.



- A **FleetManager** owns its fleet of Drones. If the manager is shut down, its Drones are part of that shutdown. Its `__init__` method should create a dictionary to store and manage the lifecycle of its Drone objects. It should also have a list to track **pendingPackages**.

Simulation: Implement **FleetManager.dispatchJobs()**, which assigns packages to idle drones. Also, implement **simulationTick()**, which loops through all drones and deterministically updates their state (e.g., a 'delivering' drone with a timer of 2 will become 'charging' after 2 ticks).

Question 04:

A computer vision team at **FAST NUCES Karachi** needs to "augment" their image data (e.g., **flip**, **brighten**) to train a better AI. You will build a flexible "pipeline" that applies a series of transformations. You will simulate an image as a 2D list of pixel values.

Topics: OOP (Classes, Objects), Functions as Arguments, lambda, List Comprehensions.

```
# A sample 2x3 image represented as a 2D list
originalPixels = [
    [10, 20, 30],
    [40, 50, 60]
]
```

Tasks:

Image Class: Create an **Image** class that stores a 2D list of pixels. It must have an **applyTransformation(transformationFunc)** method that accepts a function as an argument and uses it to update its own pixel data.

Manual Deep Copy: Transformations must not change the original image. Implement a **getCopy()** method for your **Image** class without using the copy module. (Hint: a list comprehension can create a new, independent 2D list).



National University of Computer & Emerging Sciences, Karachi

School of Computing – BAI-3A

Fall 2025

Transformation Functions: Write standalone functions that each take a `pixelData` list and return a new 2D list:

- `flipHorizontal(pixelData)`
- `adjustBrightness(pixelData, brightnessValue)`: This one needs an extra argument.
- `rotateNinetyDegrees(pixelData)`

Pipeline Class: Create an `AugmentationPipeline` class. It should `addStep(transformFunc)` to store a list of functions. Its `processImage(originalImage)` method should loop through this list, apply each function to a fresh copy of the `originalImage`, and return a new list of all the augmented images.

Using lambda: The `adjustBrightness` function needs two arguments, but the pipeline's `applyTransformation` method only passes one. When you add this step to your pipeline, use a lambda function to wrap the `adjustBrightness` call, "pre-loading" it with the `brightnessValue` you want.